Практика: АЦП (ADC) на ESP-IDF v5.5.4 (Eclipse)

- Перед стартом: що таке `adc_oneshot_read()` і навіщо він нам (пояснення “на пальцях”)
  - Що це таке
  - Аналогія: лінійка з поділками, а не безперервна стрічка
  - Навіщо він нам потрібен (мета)
  - Як він працює “під капотом” (коротко)
  - Підсумок одним реченням
- Мета практики
- Схема підключення
- Частина 0. Налаштування проекту в Eclipse (15 хв)
- Частина 1. Перше читання АЦП: сире число замість напруги (20 хв)
  - Завдання 1.1
  - Завдання 1.2
  - Детально: що таке `ADC_ATTEN_DB_12` і навіщо він потрібен
- Частина 2. Дискретизація і квантування наочно (25 хв)
  - Завдання 2.1
  - Завдання 2.2
  - Завдання 2.3 — запитання для обговорення
- Частина 3. Калібрування у Вольти (30 хв)
  - Завдання 3.1
  - Завдання 3.2
  - Завдання 3.3 — запитання для обговорення
- Частина 4. Фінал: оверсемплінг + гістерезис (35 хв)
  - Завдання 4.1
  - Завдання 4.2
  - Завдання 4.3 — запитання для обговорення
  - Завдання 4.4 (самостійно)
- Частина 5. Термістор NTC: LDR-сигналізатор + NTC з обробкою помилок (25 хв)
  - Схема підключення
  - Завдання 5.1
  - Завдання 5.2 — запитання для обговорення
  - Завдання 5.3 — запитання для обговорення
  - Завдання 5.4 (самостійно)
  - Завдання 5.5 (самостійно) — сигналізатор перегріву
- Додаток A. Порівняння з Arduino-кодом уроку
- Додаток B. Чому ADC1, а не ADC2
- Додаток C. Формули: LSB, похибка квантування, ефективні біти оверсемплінгу

Практика: АЦП (ADC) на ESP-IDF v5.5.4 (Eclipse)

Тривалість: 2 год 25 хв (Частини 1–4 — база; Частина 5 з термістором — розширення) **Плата:** ESP32 (будь-яка devkit-плата) **Середовище:** Espressif-IDE (Eclipse) на базі ESP-IDF v5.5.4

Перед стартом: що таке `adc_oneshot_read()` і навіщо він нам (пояснення “на пальцях”)

Ця функція буде з’являтися майже в кожній частині практики, тож розберемо її одразу, простими словами, ще до коду.

Що це таке

`adc_oneshot_read()` — це функція, яка бере **один замір** напруги на обраному аналоговому піні й повертає **ціле число** — індекс дискретного рівня, найближчого до реальної напруги.

```
int raw = 0;
adc_oneshot_read(adc1_handle, ADC_CHANNEL_3, &raw); // наприклад, поверне 2048 — це "сире" число АЦП
```

Це просто одне ціле число від 0 до 4095 (при 12-бітній роздільній здатності). Саме по собі воно ще не є вольтами — це індекс одного з 4096 можливих рівнів, на які АЦП поділив діапазон опорної напруги.

Аналогія: лінійка з поділками, а не безперервна стрічка

Уявіть звичайну лінійку, поділену на міліметри. Довжина предмета в реальності — неперервна (може бути 37.428 мм), але лінійка змушує округлити до найближчої поділки — скажімо, 37 мм. АЦП робить те саме з напругою: неперервний сигнал (будь-яке значення від 0 до 3.3 В) він округлює до найближчої з 4096 “поділок”.

`adc_oneshot_read()` — це не “виміряй мені точну напругу”. Це команда **“подивись, на яку поділку лінійки найближче лягає поточна напруга, і скажи номер цієї поділки”**.

Навіщо він нам потрібен (мета)

Головна ідея АЦП-перетворення з лекції — міст між аналоговим світом (неперервна напруга з датчика) і цифровим світом (числа, з якими вміє працювати процесор). Щоб цей міст працював, потрібні три кроки:

1. **Дискретизація** — зафіксувати напругу в конкретний момент часу (а не намагатись зберегти її неперервно).
2. **Квантування** — округлити цю напругу до найближчого з дискретних рівнів (поділок “лінійки”).
3. **Кодування** — видати номер цього рівня як звичайне ціле число, зрозуміле програмі.

Виклик `adc_oneshot_read()` виконує всі три кроки одразу, за одне звернення. У цій практиці ми розберемо кожен крок окремо (через експерименти з частотою читання і роздільною здатністю), а потім зберемо їх у повноцінний робочий пристрій.

Як він працює “під капотом” (коротко)

Всередині ESP32 є апаратний блок SAR-АЦП (Successive Approximation Register) — послідовно порівнює вхідну напругу з внутрішньою “еталонною драбинкою” напруг, звужуючи діапазон вдвічі на кожному кроці порівняння, поки не визначить найближчий рівень. Для 12-бітного результату потрібно 12 таких порівнянь — усе це відбувається за одиниці мікросекунд, апаратно, без участі основного коду.

> Детальніші формули (LSB, похибка квантування, чому оверсемплінг додає ефективні біти) — у **Додатку С** наприкінці файлу.

Підсумок одним реченням

`adc_oneshot_read()` — це “подивись на поділку лінійки, куди найближче лягла напруга, і скажи її номер”, і саме на цій ідеї — дискретизація, квантування, кодування — побудована вся практика: від першого сирого числа до готового світлового сигналізатора з калібруванням і оверсемплінгом.

Мета практики

За два уроки ми пройдемо шлях від “сирого” числа до професійного вимірювання:

1. Побачити саме перетворення наживо: одне число АЦП замість неперервної напруги
2. Виміряти вплив частоти дискретизації і розрядності на якість даних
3. Перевести число у вольти правильно — із заводським калібруванням чипа
4. Побудувати неблокуючий детектор порогу (світло/темрява) з оверсемплінгом і гістерезисом — аналоговий аналог дебаунсу
5. Додати другий незалежний датчик (термістор NTC) на окремому каналі того самого ADC1 і переконатись, що АЦП-логіка не залежить від типу датчика — лише формула перерахунку відрізняється

Усі приклади написані на чистому ESP-IDF (C) з новим API `esp_adc/adc_oneshot.h`, без Arduino-обгортки — бо ми працюємо в Eclipse/ESP-IDF, а не в Arduino IDE.

Схема підключення

Компонент	GPIO	Примітка
Дільник напруги: LDR + резистор 10 кОм	GPIO34	`3.3V → LDR → GPIO34 → резистор 10 кОм → GND`. На світлі опір LDR падає → напруга у вузлі росте
Світлодіод	GPIO5	Через резистор ~330 Ом на GND — сигналізатор темряви

GPIO34 на ESP32 (класичний, не S3) відповідає каналу **ADC1_CHANNEL_6**. АЦП1 не конфліктує з Wi-Fi, тому саме його і використовуємо для датчика (на відміну від АЦП2, який стає недоступним, щойно активне радіо).

> Якщо у вас плата ESP32-S3 — канали ADC1 інші (GPIO1–10 → CHANNEL_0–9), звірте з даташитом вашої плати й підставте правильний `ADC_CHANNEL_x` в коді нижче.

Частина 0. Налаштування проекту в Eclipse (15 хв)

1. **File → New → Espressif IDF Project**
2. Назва проекту: `adc_lab`
3. Оберіть target: `esp32`
4. У `main/CMakeLists.txt` переконайтесь, що `main.c` є у списку `SRCS`
5. **Пропишіть залежності (компоненти) в `main/CMakeLists.txt`** — без цього збірка впаде з помилкою `fatal error: esp_adc/adc_oneshot.h: No such file or directory`:

```
idf_component_register(
    SRCS "main.c"
    INCLUDE_DIRS "."
    REQUIRES driver esp_adc esp_timer log freertos
)
```

Компонент	Навіщо потрібен	Використовується в
`driver`	`driver/gpio.h` — керування світлодіодом	усі частини
`esp_adc`	`esp_adc/adc_oneshot.h`, `esp_adc/adc_cali.h` — увесь АЦП-функціонал	усі частини
`esp_timer`	`esp_timer.h` — `esp_timer_get_time()` для вимірювання часу	Частини 2, 4
`log`	`esp_log.h` — `ESP_LOGI()`	усі частини
`freertos`	`freertos/FreeRTOS.h`, `freertos/task.h`	усі частини

> У ESP-IDF v5.x компоненти `freertos` і `log` часто підтягуються транзитивно, але **явно вказувати їх — хороша практика**: план збірки лишається прозорим для читання.

6. Відкрийте `sdkconfig` через **ESP-IDF: SDK Configuration Editor**, переконайтесь, що вибрано правильний Serial Port (Flash → Serial flasher config)
7. Зробіть перший build: кнопка з молоточком (Build) на панелі інструментів
8. Прошийте та відкрийте монітор: **Run → Run As → ESP-IDF Application** (або кнопки Flash / Monitor)

Чек-поінт: у моніторі повинен з’явитись стандартний лог завантаження ESP-IDF без помилок.

Частина 1. Перше читання АЦП: сире число замість напруги (20 хв)

Спочатку просто читаємо “сире” значення й друкуємо його — щоб побачити, що АЦП повертає завжди ціле число, а не плавну величину.

```c
#include <stdio.h>              // стандартна бібліотека C
#include "freertos/FreeRTOS.h"  // базові типи й макроси FreeRTOS (потрібен для vTaskDelay, pdMS_TO_TICKS)
#include "freertos/task.h"      // функції керування задачами: vTaskDelay і т.п.
#include "esp_adc/adc_oneshot.h" // новий (IDF v5.x) API одноразових замірів АЦП
#include "esp_log.h"            // макрос ESP_LOGI() для виводу в UART-монітор

#define ADC_UNIT     ADC_UNIT_1        // АЦП1 — не конфліктує з Wi-Fi
#define ADC_CHANNEL  ADC_CHANNEL_6     // GPIO34 на класичному ESP32 відповідає ADC1_CHANNEL_6
static const char *TAG = "ADC_LAB";    // "тег" логів — буде показуватись перед кожним повідомленням у моніторі

void app_main(void)             // точка входу програми на ESP-IDF (аналог setup()+loop() в Arduino, але одна функція)
{
    adc_oneshot_unit_handle_t adc1_handle;                // хендл (дескриптор) блока АЦП1 — потрібен для всіх подальших викл
    adc_oneshot_unit_init_cfg_t init_config = {
        .unit_id = ADC_UNIT,                              // який саме блок АЦП ініціалізуємо
    };
    adc_oneshot_new_unit(&init_config, &adc1_handle);     // створюємо (реєструємо) блок АЦП1 і отримуємо хендл

    adc_oneshot_chan_cfg_t chan_config = {
        .bitwidth = ADC_BITWIDTH_DEFAULT,                 // роздільна здатність — типова для чипа (12 біт на класичному ESP3
        .atten    = ADC_ATTEN_DB_12,                      // послаблення входу — дозволяє читати повний діапазон ~0-3.3В (дет
    };
    adc_oneshot_config_channel(adc1_handle, ADC_CHANNEL, &chan_config); // прив'язуємо налаштування до конкретного каналу (піна

    while (1) {                                           // нескінченний цикл — аналог loop() в Arduino, виконується постійно
        int raw = 0;                                      // сюди прийде "сире" число АЦП
        adc_oneshot_read(adc1_handle, ADC_CHANNEL, &raw);  // ОДИН замір: дискретизація + квантування + кодування за один викл
        ESP_LOGI(TAG, "Сире значення: %d", raw);          // друкуємо в монітор

        vTaskDelay(pdMS_TO_TICKS(200));                   // пауза 200 мс — щоб монітор встигав, і задача не займала весь CPU
    }
}
```

Завдання 1.1

Прошийте, посвітіть ліхтариком телефону на датчик, потім прикрийте його рукою. Запишіть приблизний діапазон значень, які ви побачили (мінімум і максимум).

Завдання 1.2

Поясніть своїми словами (у коментарі в коді або усно): чому `raw` — це саме ціле число від 0 до 4095, а не, скажімо, `3.14` чи `2.718`, попри те, що реальна напруга на піні змінюється плавно.

> **Примітка про різницю з Arduino:** `analogRead()` в Arduino-обгортці ESP32 приховує від вас усю ініціалізацію (`adc_oneshot_new_unit` , `adc_oneshot_config_channel`) — вона виконується один раз автоматично при першому виклику. У “чистому” ESP-IDF ці кроки треба зробити явно, зате ви точно бачите й контролюєте кожен параметр АЦП.

Детально: що таке `ADC_ATTEN_DB_12` і навіщо він потрібен

Цей параметр зустрічається в кожній частині практики — розберемо його один раз докладно.

Проблема: внутрішній АЦП ESP32 фізично здатен коректно вимірювати напругу лише в діапазоні приблизно **0–1.1 В** (це його “рідний” робочий діапазон). Але датчики (і наш дільник напруги) видають напругу до **3.3 В**. Без додаткових заходів усе, що вище 1.1 В, АЦП вимірював би неправильно (сигнал “обрізався б”).

Рішення — атенюатор (attenuation, “послаблення входу”): апаратний блок перед самим АЦП, який **пропорційно зменшує** вхідну напругу перед вимірюванням, а потім результат перераховується назад. Це як переставити лінійку — та сама кількість поділок (4096), але тепер вони покривають ширший діапазон реальної напруги.

Атенюація	Приблизний робочий діапазон входу	Коли використовувати
`ADC_ATTEN_DB_0`	0 – 1.1 В	сигнал заздалегідь звужений до 1.1В (рідко)
`ADC_ATTEN_DB_2_5`	0 – 1.5 В	проміжні випадки

Атенюація	Приблизний робочий діапазон входу	Коли використовувати
`ADC_ATTEN_DB_6`	0 – 2.2 В	датчики з живленням ~2В
`ADC_ATTEN_DB_12`	0 – ~3.3 В (реально трохи менше, до ~3.1В)	**наш випадок**: дільник напруги від 3.3В

Чому саме `DB_12` у нашій практиці: дільник напруги LDR+резистор живиться від 3.3 В, тож напруга на вузлі може сягати значень, близьких до 3.3 В. Лише з `ADC_ATTEN_DB_12` АЦП здатен коректно охопити весь цей діапазон без обрізання зверху.

Побічний ефект: що більше послаблення, то трохи **менша точність** біля нижньої межі діапазону (те саме число поділок розтягнуто на ширший діапазон вольтів → кожна поділка “вартує” більше мілівольт). Це прямий наслідок формули LSB = Vref/2^N з лекції — просто тут “Vref” фактично розширюється атенюацією.

Частина 2. Дискретизація і квантування наочно (25 хв)

Розберемо окремо, як частота читання (дискретизація) і розрядність (квантування) впливають на дані — саме так, як описано в лекції.

```c
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_adc/adc_oneshot.h"
#include "esp_log.h"

#define ADC_UNIT     ADC_UNIT_1
#define ADC_CHANNEL  ADC_CHANNEL_6

static const char *TAG = "ADC_LAB";

void app_main(void)
{
    adc_oneshot_unit_handle_t adc1_handle;
    adc_oneshot_unit_init_cfg_t init_config = { .unit_id = ADC_UNIT };
    adc_oneshot_new_unit(&init_config, &adc1_handle);

    adc_oneshot_chan_cfg_t chan_config = {
        .bitwidth = ADC_BITWIDTH_9,       // ← КВАНТУВАННЯ: навмисно занижена розрядність (512 рівнів замість 4096)
        .atten    = ADC_ATTEN_DB_12,
    };
    adc_oneshot_config_channel(adc1_handle, ADC_CHANNEL, &chan_config);

    while (1) {
        int raw = 0;
        adc_oneshot_read(adc1_handle, ADC_CHANNEL, &raw);
        ESP_LOGI(TAG, "raw(9-bit)=%d", raw);

        // ← ДИСКРЕТИЗАЦІЯ: спробуй три значення затримки по черзі
        vTaskDelay(pdMS_TO_TICKS(20));       // ① часта дискретизація (~50 Гц)
        // vTaskDelay(pdMS_TO_TICKS(200));    // ② помірна (~5 Гц)
        // vTaskDelay(pdMS_TO_TICKS(1000));   // ③ рідка (~1 Гц)
    }
}
```

`ADC_BITWIDTH_9` доступний на ESP32 у діапазоні `ADC_BITWIDTH_9` … `ADC_BITWIDTH_12` (залежно від чипа — на деяких моделях лінійка інша, звіряйте з `adc_types.h`).

Завдання 2.1

Залийте з `ADC_BITWIDTH_9` і `vTaskDelay(20)`. Повільно перекривайте датчик рукою і спостерігайте за логом — порівняйте плавність зі значеннями з Частини 1 (де була 12-бітна за замовчуванням розрядність). Опишіть різницю: чи виглядають зміни “сходинками”?

Завдання 2.2

Поверніть `ADC_BITWIDTH_DEFAULT` (12 біт), але тепер поекспериментуйте із затримкою: `20` мс, потім `1000` мс. Швидко махайте рукою над датчиком у кожному режимі. При якій затримці ви помічаєте, що швидкі рухи “губляться” або показник змінюється зовсім не так, як реальний рух руки? Зв’яжіть спостереження з теоремою Nyquist із лекції.

Завдання 2.3 — запитання для обговорення

Чому для повільного датчика (кімнатна температура) було б марнотратно ставити `vTaskDelay(1)` (тобто читати з частотою ~1000 Гц)? *(Підказка: подумайте про теорему Nyquist "у зворотний бік" — навіщо частота дискретизації, набагато вища за необхідну.)*

Частина 3. Калібрування у Вольти (30 хв)

У Частині 1 ми отримали "сире" ціле число. Тепер переведемо його у вольти — причому не приблизно (`raw/4095*3.3`), а з використанням **заводського калібрування** конкретного чипа, яке набагато точніше враховує реальні характеристики АЦП.

```c
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_adc/adc_oneshot.h"
#include "esp_adc/adc_cali.h"           // новий інклюд: типи й функції калібрування
#include "esp_adc/adc_cali_scheme.h"    // новий інклюд: конкретна схема калібрування (curve fitting / line fitting)
#include "esp_log.h"

#define ADC_UNIT     ADC_UNIT_1
#define ADC_CHANNEL  ADC_CHANNEL_6

static const char *TAG = "ADC_CALI";

void app_main(void)
{
    adc_oneshot_unit_handle_t adc1_handle;
    adc_oneshot_unit_init_cfg_t init_config = { .unit_id = ADC_UNIT };
    adc_oneshot_new_unit(&init_config, &adc1_handle);

    adc_oneshot_chan_cfg_t chan_config = {
        .bitwidth = ADC_BITWIDTH_DEFAULT,   // назад до повних 12 біт
        .atten    = ADC_ATTEN_DB_12,
    };
    adc_oneshot_config_channel(adc1_handle, ADC_CHANNEL, &chan_config);

    // --- НОВЕ: створюємо об'єкт калібрування ---
    adc_cali_handle_t cali_handle = NULL;
    adc_cali_curve_fitting_config_t cali_config = {
        .unit_id  = ADC_UNIT,
        .chan     = ADC_CHANNEL,
        .atten    = ADC_ATTEN_DB_12,
        .bitwidth = ADC_BITWIDTH_DEFAULT,
    };
    esp_err_t ret = adc_cali_create_scheme_curve_fitting(&cali_config, &cali_handle); // читає заводські дані з eFuse чипа

    if (ret != ESP_OK) {
        ESP_LOGI(TAG, "Заводське калібрування недоступне на цьому чипі, використовуємо приблизну формулу");
    }

    while (1) {
        int raw = 0;
        adc_oneshot_read(adc1_handle, ADC_CHANNEL, &raw);

        int voltage_mv_calibrated = 0;
        adc_cali_raw_to_voltage(cali_handle, raw, &voltage_mv_calibrated); // ТОЧНЕ переведення з урахуванням eFuse-даних чипа

        float voltage_mv_naive = (raw / 4095.0f) * 3300.0f;              // ПРИБЛИЗНЕ переведення "на око" (як у лекції)

        ESP_LOGI(TAG, "raw=%4d   calibrated=%d mV   naive=%.0f mV   diff=%d mV",
                 raw, voltage_mv_calibrated, voltage_mv_naive,
                 voltage_mv_calibrated - (int)voltage_mv_naive);

        vTaskDelay(pdMS_TO_TICKS(300));
    }
}
```

Завдання 3.1

Прошийте і подивіться на стовпчик `diff` — наскільки заводське калібрування (`calibrated`) відрізняється від "наївної" формули (`naive`)? Зазвичай розбіжність — десятки мілівольт.

Завдання 3.2

Виміряйте реальну напругу на GPIO34 мультиметром і порівняйте з обома стовпчиками — `calibrated` і `naive`. Який ближче до показу мультиметра?

Завдання 3.3 — запитання для обговорення

У лекції сказано: “справжня Vref на ESP32 часто не 3.3 В”. Поясніть своїми словами, чому заводське калібрування (`adc_cali_raw_to_voltage`) вирішує цю проблему точніше, ніж просто виміряти живлення мультиметром один раз і підставити число у формулу вручну. *(Підказка: подумайте не лише про Vref, а й про нелінійність самої характеристики АЦП по всьому діапазону.)*

Частина 4. Фінал: оверсемплінг + гістерезис (35 хв)

Зберемо все докупи в єдиний неблокуючий пристрій: періодичну дискретизацію, оверсемплінг проти шуму, калібровані вольти й пороговий детектор із **гістерезисом** — амплітудним аналогом дебаунсу з попереднього модуля.

```c
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_adc/adc_oneshot.h"
#include "esp_adc/adc_cali.h"
#include "esp_adc/adc_cali_scheme.h"
#include "esp_timer.h"          // esp_timer_get_time() — для неблокуючого розкладу читань
#include "esp_log.h"

#define ADC_UNIT       ADC_UNIT_1
#define ADC_CHANNEL    ADC_CHANNEL_6
#define LED_PIN        GPIO_NUM_5

#define SAMPLES            16          // оверсемплінг: скільки замірів усереднюємо за один цикл
#define SAMPLE_PERIOD_US   (50 * 1000) // період дискретизації: 50 мс = 20 Гц
#define THRESHOLD_ON_RAW   1500        // стає темно нижче цього "сирого" рівня
#define THRESHOLD_OFF_RAW  1800        // вважаємо "знову світло" вище цього рівня

static const char *TAG = "ADC_FINAL";

// Один усереднений замір — оверсемплінг проти шуму (Частина 4 лекції)
static int read_averaged(adc_oneshot_unit_handle_t handle)
{
    long sum = 0;
    for (int i = 0; i < SAMPLES; i++) {
        int raw = 0;
        adc_oneshot_read(handle, ADC_CHANNEL, &raw);
        sum += raw;
    }
    return (int)(sum / SAMPLES);
}

void app_main(void)
{
    // --- налаштування виходу LED ---
    gpio_config_t led_conf = {
        .pin_bit_mask = 1ULL << LED_PIN,
        .mode = GPIO_MODE_OUTPUT,
    };
    gpio_config(&led_conf);

    // --- налаштування АЦП ---
    adc_oneshot_unit_handle_t adc1_handle;
    adc_oneshot_unit_init_cfg_t init_config = { .unit_id = ADC_UNIT };
    adc_oneshot_new_unit(&init_config, &adc1_handle);

    adc_oneshot_chan_cfg_t chan_config = {
        .bitwidth = ADC_BITWIDTH_DEFAULT,
        .atten    = ADC_ATTEN_DB_12,
    };
    adc_oneshot_config_channel(adc1_handle, ADC_CHANNEL, &chan_config);

    // --- калібрування (для друку вольтів у лог, самій логіці порогу не потрібне) ---
    adc_cali_handle_t cali_handle = NULL;
```

```
    adc_cali_curve_fitting_config_t cali_config = {
        .unit_id = ADC_UNIT, .chan = ADC_CHANNEL,
        .atten = ADC_ATTEN_DB_12, .bitwidth = ADC_BITWIDTH_DEFAULT,
    };
    adc_cali_create_scheme_curve_fitting(&cali_config, &cali_handle);

    bool led_on = false;               // офіційний, підтверджений гістерезисом стан
    int64_t last_sample = 0;           // мітка часу останнього виміру (мкс)

    while (1) {
        int64_t now = esp_timer_get_time();

        if (now - last_sample >= SAMPLE_PERIOD_US) {   // неблокуюча дискретизація за розкладом
            last_sample = now;

            int level = read_averaged(adc1_handle);    // квантоване, усереднене значення

            int mv = 0;
            adc_cali_raw_to_voltage(cali_handle, level, &mv);

            // --- гістерезис: два різні пороги замість одного (аналоговий аналог дебаунсу) ---
            if (!led_on && level < THRESHOLD_ON_RAW) {
                led_on = true;                          // стало темно → вмикаємо
            } else if (led_on && level > THRESHOLD_OFF_RAW) {
                led_on = false;                         // стало досить світло → вимикаємо
            }
            gpio_set_level(LED_PIN, led_on);

            ESP_LOGI(TAG, "level=%4d (%d mV)   led=%s", level, mv, led_on ? "ON" : "off");
        }

        vTaskDelay(pdMS_TO_TICKS(5)); // невелика пауза — лишає час планувальнику FreeRTOS, не впливає на точність дискретизац
    }
}
```

Завдання 4.1

Прошийте і переконайтесь, що LED вмикається в темряві та вимикається на світлі, **без мерехтіння** на межі.

Завдання 4.2

Замініть `THRESHOLD_ON_RAW` і `THRESHOLD_OFF_RAW` на **однакове** значення (наприклад, `1650` для обох). Плавно тримайте долоню на межі тінь/світло над датчиком. Порахуйте, скільки разів LED “клацнув” за 10 секунд. Поверніть різні пороги — і порахуйте знову. Порівняйте результати.

Завдання 4.3 — запитання для обговорення

Поясніть, чому гістерезис тут — це **той самий принцип**, що й часовий дебаунс кнопки з попереднього модуля, хоча технічно це зовсім інший механізм (амплітудний поріг, а не часове вікно). Що спільного в самій ідеї?

Завдання 4.4 (самостійно)

Змініть `SAMPLES` з `16` на `1` (тобто вимкніть оверсемплінг) і подивіться, чи почне LED частіше “сіпатись” на межі порогу навіть із гістерезисом. Поясніть, чому оверсемплінг і гістерезис вирішують **різні** проблеми (шум одиничного заміру проти коливань освітлення навколо порогу), хоча обидва зменшують “мерехтіння”.

Частина 5. Термістор NTC: LDR-сигналізатор + NTC з обробкою помилок (25 хв)

У вас у наборі є і фоторезистор, і термістор — тож підключимо обидва одночасно, кожен на своєму каналі **ADC1**, у повноцінній, протестованій на залізі програмі. Це вже не короткий навчальний фрагмент, а завершений `main.c`: LDR керує сигналізатором темряви (з калібруванням і гістерезисом, як у Частині 4), а NTC паралельно друкує температуру — і кожен виклик АЦП тут **перевіряє код повернення**, а не сліпо довіряє результату.

Схема підключення

Компонент	GPIO	Примітка
Дільник напруги: LDR + резистор 10 кОм	GPIO33	`3.3V → LDR → GPIO33 → резистор 10 кОм → GND`. Канал `ADC1_CHANNEL_5`

Компонент	GPIO	Примітка
Дільник напруги: NTC + резистор 10 кОм	GPIO39	`3.3V → NTC → GPIO39 → резистор 10 кОм → GND`. Канал `ADC1_CHANNEL_3`, вхід-only, без внутрішніх підтяжок
Світлодіод	GPIO5	Через резистор ~330 Ом на GND — сигналізатор темряви (той самий LED, що й у Частині 4)

> **Це самостійна фінальна програма**, а не доповнення до Частини 4 — вона сама ініціалізує GPIO, АЦП і калібрування з нуля. Якщо раніше LDR стояв у вас на `GPIO34`, для цієї версії просто перепаяйте провід дільника на `GPIO33` (сусідній вхід-only пін ADC1). Фізично сенсор той самий — змінюється лише конкретний канал, обраний для цього прикладу.

Термістор — компонент **без полярності** (як звичайний резистор): переплутати виводи місцями неможливо.

```c
#include <stdio.h>
#include <stdbool.h>
#include <unistd.h>
#include <math.h>

#include "driver/gpio.h"
#include "esp_adc/adc_cali.h"
#include "esp_adc/adc_cali_scheme.h"
#include "esp_adc/adc_oneshot.h"
#include "esp_err.h"
#include "esp_log.h"
#include "esp_timer.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "hal/adc_types.h"

#define LDR_UNIT        ADC_UNIT_1
#define LDR_CHANNEL     ADC_CHANNEL_5      // GPIO33
#define NTC_UNIT        ADC_UNIT_1
#define NTC_CHANNEL     ADC_CHANNEL_3      // GPIO39
#define LED_PIN         GPIO_NUM_5

#define SAMPLES                 16
#define SAMPLE_PERIOD_US        (50 * 1000)
#define THRESHOLD_ON_RAW        1500
#define THRESHOLD_OFF_RAW       1800

#define ADC_MAX_RAW             4095
#define APPROX_ADC_FULL_MV      3300

/* NTC datasheet values:
 * R25      = 10 kohm +/-1%
 * B25/50   = 3950 K +/-1%
 *
 * Divider connection must match the DigiKey example:
 * 3.3 V -> NTC -> ADC node -> SERIES_R -> GND
 */
static const float SERIES_R_OHM = 10000.0f;
static const float NTC_R25_OHM  = 10000.0f;
static const float NTC_T25_C    = 25.0f;
static const float NTC_B25_50_K = 3950.0f;

static const char *TAG = "NTC";

static float raw_to_ntc_resistance_ohm(int raw)
{
    if (raw <= 0 || raw >= ADC_MAX_RAW) {
        return NAN;                         // захист від ділення на нуль і межових "сирих" значень
    }
```

```
    /* DigiKey divider formula:
     * R_NTC = R_series * (ADC_max / ADC_raw - 1)
     */
    return SERIES_R_OHM *
           ((float)ADC_MAX_RAW / (float)raw - 1.0f);
}

static float raw_to_temperature_c(int raw)
{
    const float r_ntc = raw_to_ntc_resistance_ohm(raw);

    if (!isfinite(r_ntc) || r_ntc <= 0.0f) {
        return NAN;                         // NAN з попередньої функції поширюємо далі, а не тихо рахуємо сміття
    }

    const float t25_k = NTC_T25_C + 273.15f;

    /* Beta equation:
     * 1/T = 1/T25 + ln(R/R25)/B
     */
    const float inverse_t =
        (1.0f / t25_k) +
        (logf(r_ntc / NTC_R25_OHM) / NTC_B25_50_K);

    if (!isfinite(inverse_t) || inverse_t <= 0.0f) {
        return NAN;
    }

    return (1.0f / inverse_t) - 273.15f;
}

static int read_averaged(adc_oneshot_unit_handle_t handle,
                         adc_channel_t channel)
{
    int64_t sum = 0;
    int valid_samples = 0;                  // рахуємо лише вдалі заміри — не всі SAMPLES обов'язково пройдуть без помилок

    for (int i = 0; i < SAMPLES; i++) {
        int raw = 0;

        if (adc_oneshot_read(handle, channel, &raw) == ESP_OK) {
            sum += raw;
            valid_samples++;
        }
    }

    if (valid_samples == 0) {
        return -1;                          // сигналізуємо виклику: жодного вдалого виміру не було
    }

    return (int)(sum / valid_samples);
}

void app_main(void)
{
    gpio_config_t led_conf = {
        .pin_bit_mask = 1ULL << LED_PIN,
        .mode = GPIO_MODE_OUTPUT,
    };
    ESP_ERROR_CHECK(gpio_config(&led_conf));

    adc_oneshot_unit_handle_t adc1_handle = NULL;
    adc_oneshot_unit_init_cfg_t init_config = {
        .unit_id = LDR_UNIT,
    };
    ESP_ERROR_CHECK(adc_oneshot_new_unit(&init_config, &adc1_handle));

    adc_oneshot_chan_cfg_t chan_config = {
        .bitwidth = ADC_BITWIDTH_12,
        .atten = ADC_ATTEN_DB_12,
    };
    ESP_ERROR_CHECK(adc_oneshot_config_channel(       // один юніт ADC1 — два незалежно налаштовані канали
```

```
        adc1_handle, LDR_CHANNEL, &chan_config));
    ESP_ERROR_CHECK(adc_oneshot_config_channel(
        adc1_handle, NTC_CHANNEL, &chan_config));

    adc_cali_handle_t cali_handle = NULL;
    adc_cali_curve_fitting_config_t cali_config = {
        .unit_id = LDR_UNIT,
        .chan = LDR_CHANNEL,              // калібрування прив'язане саме до каналу LDR
        .atten = ADC_ATTEN_DB_12,
        .bitwidth = ADC_BITWIDTH_12,
    };

    const esp_err_t cali_ret =
        adc_cali_create_scheme_curve_fitting(&cali_config, &cali_handle);
    const bool calibration_available = (cali_ret == ESP_OK);

    if (!calibration_available) {
        ESP_LOGW(TAG,
                 "ADC factory calibration unavailable; "
                 "LDR voltage will be approximate");
    }

    bool led_on = false;
    int64_t last_sample = 0;

    while (true) {
        const int64_t now = esp_timer_get_time();

        if (now - last_sample >= SAMPLE_PERIOD_US) {   // неблокуюча дискретизація LDR за розкладом
            last_sample = now;

            const int level = read_averaged(adc1_handle, LDR_CHANNEL);

            if (level >= 0) {
                int voltage_mv =
                    (level * APPROX_ADC_FULL_MV) / ADC_MAX_RAW;   // "наївна" формула — запасний варіант

                if (calibration_available) {
                    int calibrated_mv = 0;
                    if (adc_cali_raw_to_voltage(
                            cali_handle, level, &calibrated_mv) == ESP_OK) {
                        voltage_mv = calibrated_mv;               // а тут — точне, якщо калібрування вдалося
                    }
                }

                if (!led_on && level < THRESHOLD_ON_RAW) {
                    led_on = true;
                } else if (led_on && level > THRESHOLD_OFF_RAW) {
                    led_on = false;
                }

                gpio_set_level(LED_PIN, led_on ? 1 : 0);

                ESP_LOGI(TAG,
                         "LDR=%4d (%d mV)\tLED=%s",
                         level,
                         voltage_mv,
                         led_on ? "ON" : "OFF");
            } else {
                ESP_LOGW(TAG, "Failed to read LDR ADC channel");  // АЦП повернув помилку — не тихо ігноруємо
            }
        }

        const int ntc_raw = read_averaged(adc1_handle, NTC_CHANNEL);  // NTC читаємо щоразу — без гейта за часом

        if (ntc_raw >= 0) {
            const float r_ntc = raw_to_ntc_resistance_ohm(ntc_raw);
            const float temp_c = raw_to_temperature_c(ntc_raw);

            if (isfinite(temp_c)) {
                ESP_LOGI(TAG,
                         "NTC=%4d\tR=%.1f ohm\tT=%.2f C",
```

```
                        ntc_raw,
                        r_ntc,
                        temp_c);
            } else {
                ESP_LOGW(TAG,
                        "Invalid NTC ADC value: raw=%d",
                        ntc_raw);
            }
        } else {
            ESP_LOGW(TAG, "Failed to read NTC ADC channel");
        }

        vTaskDelay(pdMS_TO_TICKS(500));    // темп циклу ~2 Гц — і LDR (гейт 50 мс), і NTC встигають щоразу
    }
}
```

Завдання 5.1

Прошийте і переконайтесь, що `LDR=...` і `NTC=...` друкуються в одному й тому самому циклі. Прикрийте LDR рукою й одночасно затисніть NTC між пальцями — переконайтесь, що `LED=ON` і зростання `T=...` спостерігаються незалежно одне від одного.

Завдання 5.2 — запитання для обговорення

Функція `raw_to_ntc_resistance_ohm()` явно перевіряє `raw <= 0 || raw >= ADC_MAX_RAW` і повертає `NAN`, а `raw_to_temperature_c()` одразу перевіряє `!isfinite(r_ntc)`. Поясніть своїми словами, чому саме **ділення** у формулі `SERIES_R_OHM * (ADC_MAX_RAW / raw - 1.0f)` небезпечне без цієї перевірки, і що конкретно станеться в програмі (а не в математиці), якщо `raw` дорівнюватиме `0`.

Завдання 5.3 — запитання для обговорення

На одному й тому самому `adc1_handle` виклик `adc_oneshot_config_channel()` відбувається двічі — для `LDR_CHANNEL` і для `NTC_CHANNEL`. Поясніть своїми словами, чому це можливо в межах **одного** ADC-юніта (ADC1), але неможливо було б перенести другий канал на **ADC2**, працюючи одночасно з Wi-Fi. *(Підказка: згадайте Додаток В.)*

Завдання 5.4 (самостійно)

`read_averaged()` рахує `valid_samples` окремо від загальної кількості `SAMPLES` і ділить суму саме на кількість **вдалих** вимірів. Змініть код так, щоб штучно “зіпсувати” частину вимірів (наприклад, тимчасово підмініть канал на неіснуючий), і переконайтесь, що функція коректно повертає `-1`, коли жоден замір не вдався, замість ділення на нуль.

Завдання 5.5 (самостійно) — сигналізатор перегріву

За зразком гістерезису для LDR (`THRESHOLD_ON_RAW` / `THRESHOLD_OFF_RAW`) додайте свої `THRESHOLD_ON_C` і `THRESHOLD_OFF_C` (наприклад, 28°C і 26°C) і вмикайте другий LED, коли `temp_c` перевищує поріг, вимикайте — коли опускається нижче протилежного порогу.

> **Чому формула складніша, ніж для LDR, а АЦП-частина — ні:** термістор — нелінійний компонент (опір змінюється з температурою за експонентою, а не прямо пропорційно, як у LDR). Тому пряме лінійне масштабування тут не підходить — потрібне рівняння Стейнхарта-Харта (у B-формі). Але дискретизація, квантування й кодування (сам виклик `adc_oneshot_read()`) — абсолютно ідентичні тому, що ми робили з LDR у Частинах 1-4; змінюється лише остання формула й те, наскільки ретельно код перевіряє межові випадки.

Додаток А. Порівняння з Arduino-кодом уроку

Дія	Arduino (`analogRead`)	ESP-IDF (`adc_oneshot`)
Ініціалізація АЦП	прихована, автоматична при першому виклику	явна: `adc_oneshot_new_unit()` + `adc_oneshot_config_channel()`
Один замір	`int raw = analogRead(pin);`	`adc_oneshot_read(handle, channel, &raw);`
Роздільна здатність	`analogReadResolution(12);`	поле `.bitwidth` у `adc_oneshot_chan_cfg_t`
Діапазон входу	`analogSetAttenuation(ADC_11db);`	поле `.atten` у `adc_oneshot_chan_cfg_t`

Дія	Arduino (`analogRead`)	ESP-IDF (`adc_oneshot`)
Переведення у вольти	вручну: `raw/4095.0*3.3`	`adc_cali_raw_to_voltage()` — з заводським калібруванням

Додаток B. Чому ADC1, а не ADC2

АЦП2 на ESP32 фізично використовує ту саму периферію, що й Wi-Fi-приймач для сканування каналів. Коли Wi-Fi активний (а на практиці — майже завжди, якщо у проєкті є мережа), читання з ADC2 стає непередбачуваним або взагалі недоступним. Тому в реальних проєктах для аналогових датчиків завжди обирають піни **ADC1** (на класичному ESP32 — GPIO32–39).

Додаток C. Формули: LSB, похибка квантування, ефективні біти оверсемплінгу

Крок кванта (LSB): `LSB = Vref / 2^N`, де `N` — розрядність. Для 12 біт і Vref ≈ 3.3 В: `LSB = 3.3 / 4096 ≈ 0.805 мВ`.

Максимальна похибка квантування: `± 0.5 × LSB` ≈ `± 0.4 мВ` для 12-бітного АЦП з Vref 3.3 В.

Правило оверсемплінгу: усереднення `4^n` замірів дає приблизно `+n` ефективних біт точності понад “сиру” розрядність. У Частині 4 ми усереднюємо `SAMPLES = 16 = 4²`, тобто отримуємо приблизно **+2 ефективних біти** над “сирим” 12-бітним значенням — ціна цього виграшу: 16 замірів замість одного на кожен цикл вимірювання.

Переведення мкс ↔ мс: `esp_timer_get_time()` повертає мікросекунди; `1 мс = 1000 мкс`. Саме тому в коді Частини 4 `SAMPLE_PERIOD_US` множиться на `1000` при визначенні через мілісекунди.